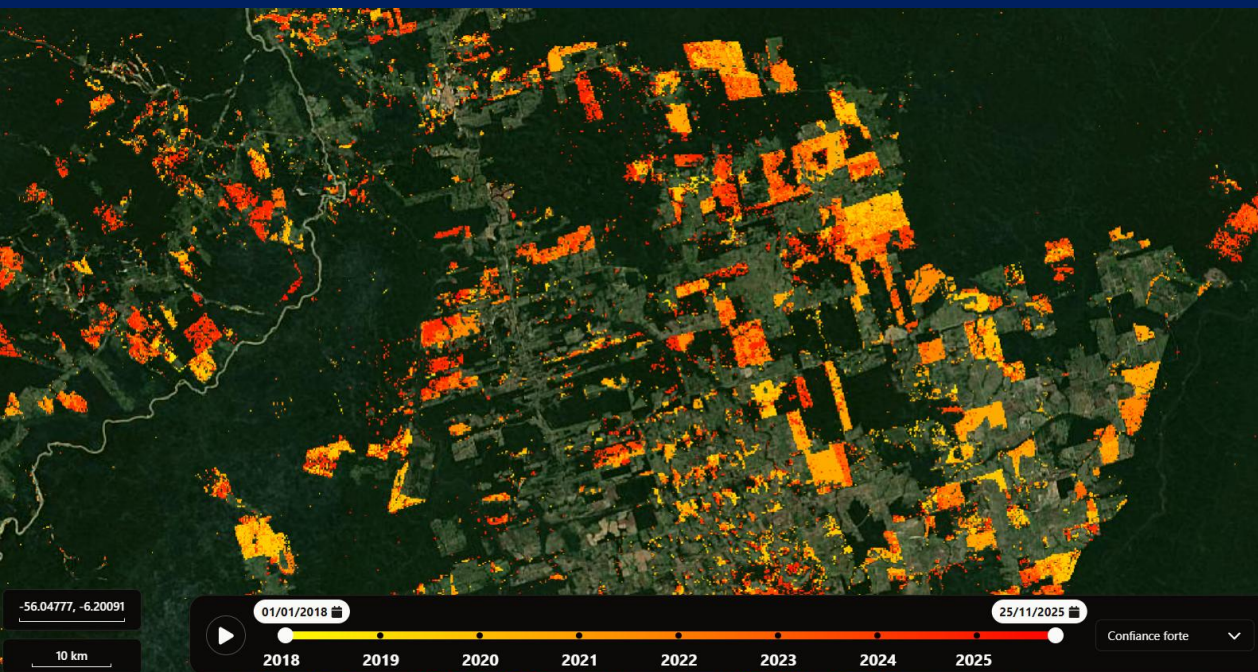


Pôle SAR

Outil détection changement de surface



Objectifs et motivations

Détecter automatiquement des changements de surface à partir d'images SAR (S1 pour le moment). Focus sur la déforestation.

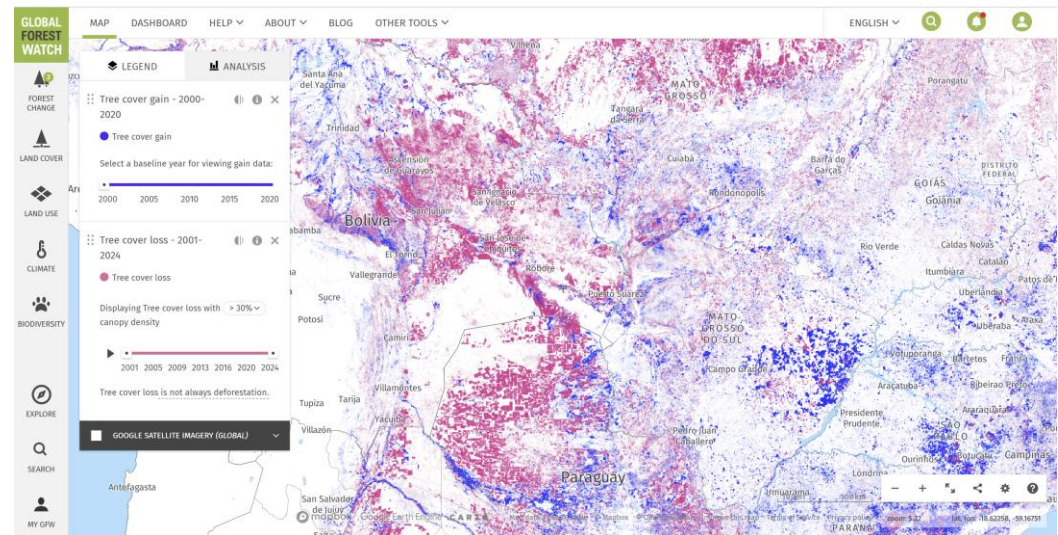
Finalité: retourner un masque binaire/shapefile, témoin des zones qui ont subi un changement

Cahier des charges :

- ✓ Automatique
- ✓ Robuste et précis
- ✓ Modulaire
- ✓ Adaptable et évolutif

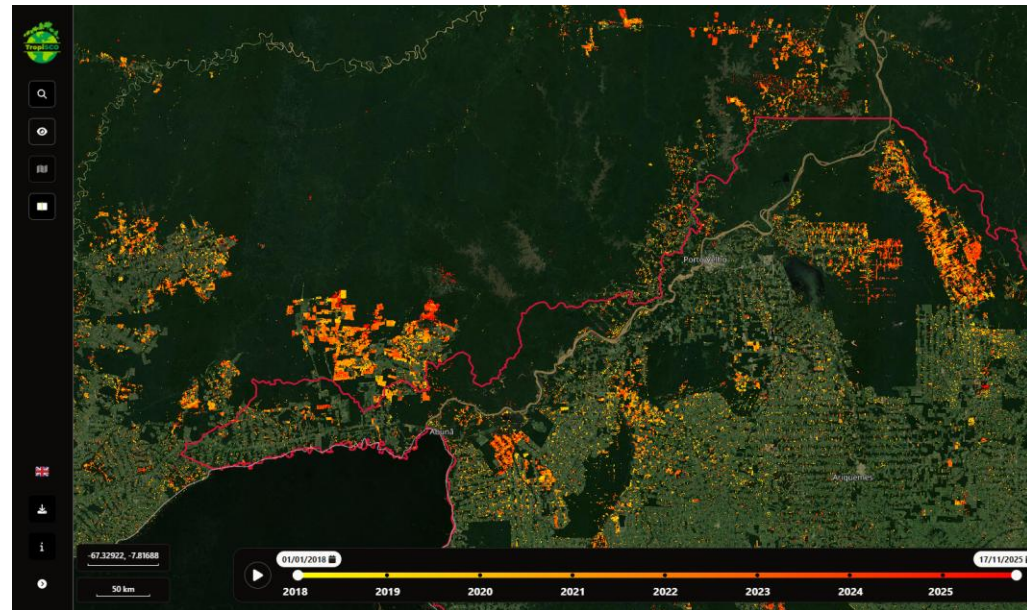


Logique simple et universelle



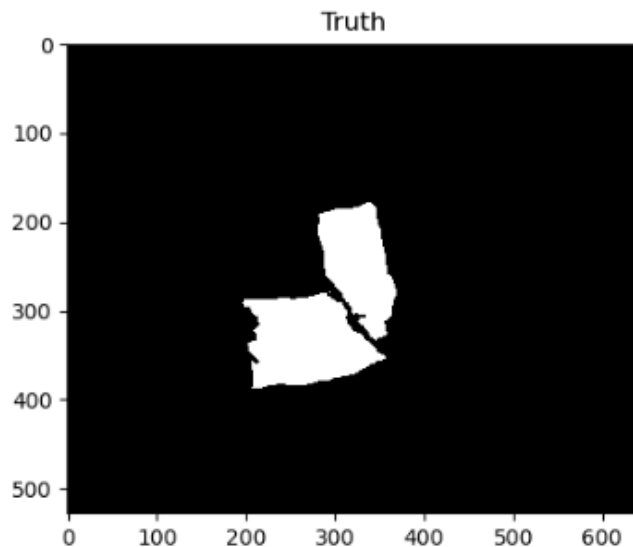
Vérité terrain

- ❖ Déforestation: surtout 3 zones tropicales : Amérique sud (bassin amazonien), Afrique centrale (bassin du Congo), Asie du Sud-Est (Indonésie, Malaisie, Papouasie-Nouvelle-Guinée). On se concentra pour l'instant sur la zone Amérique du Sud
- ❖ utilisation de Google Earth et de la plateforme Tropisco pour trouver des grandes zones d'intérêts
- ❖ Pour définir une AOI plus précisément, il faut respecter beaucoup de contraintes liées à la disponibilité des S1 et S2, aux paramètres d'acquisition des S1 et à la qualité des S2

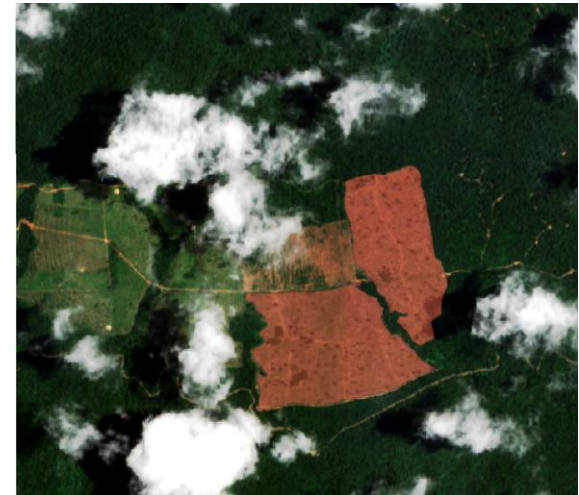


Vérité terrain

- ❖ Etablissement des shapefile de changement à la main sur QGIS à partir des S2 disponibles



- ❖ Calcul des métriques de performances entre la prédiction et le raster de vérité



- ❖ Rasterization en python, création d'un raster binaire de vérité, que l'on peut exporter si on le souhaite

```
tp : 14994
tn : 310578
fp : 9205
fn : 4727
accuracy : 0.9589636646401781
precision : 0.6196123806768619
recall : 0.7603062725013559
F1 : 0.6827868847510682
mcc : 0.6650509037427377
```

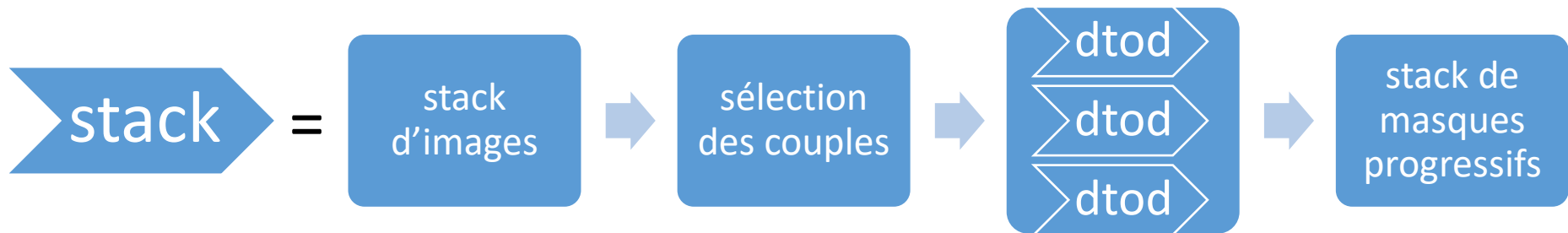
et indice kappa

Logique de l'outil

Logique de détection date – date



Logique de monitoring temps long

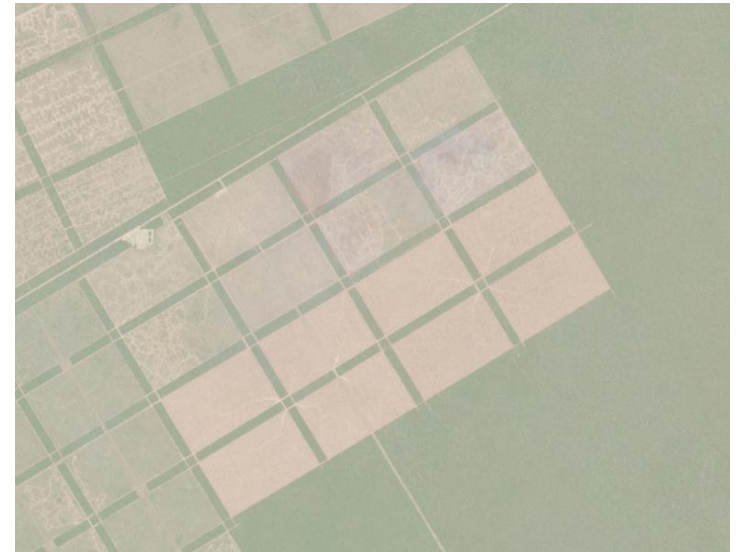
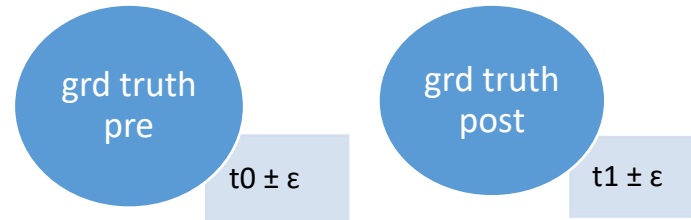


Images pre/post

❖ Sentinel 1 SLC



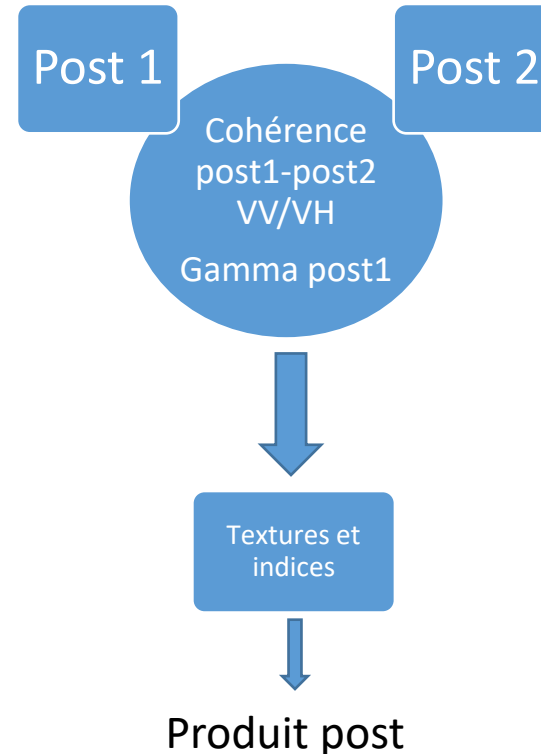
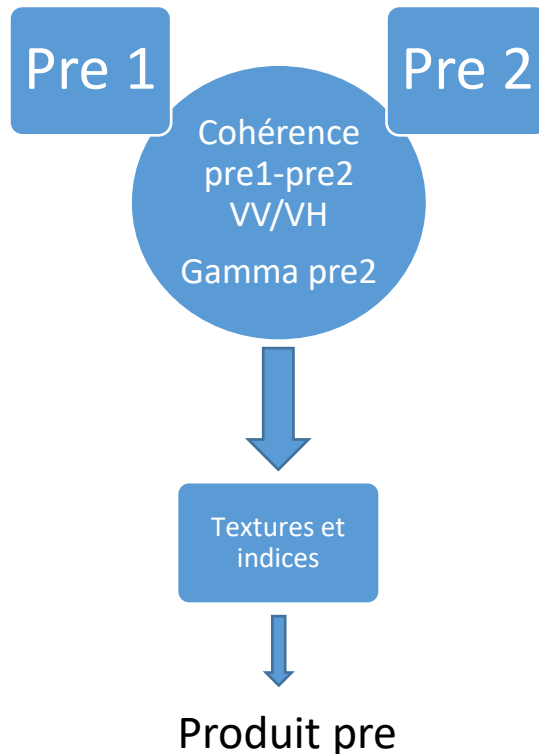
❖ Sentinel 2 (test de l'outil)



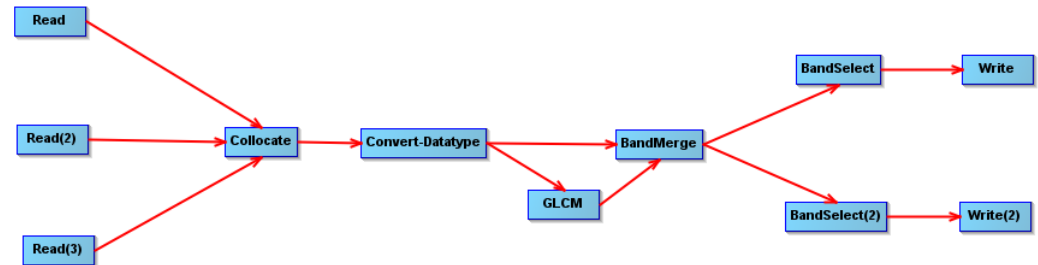
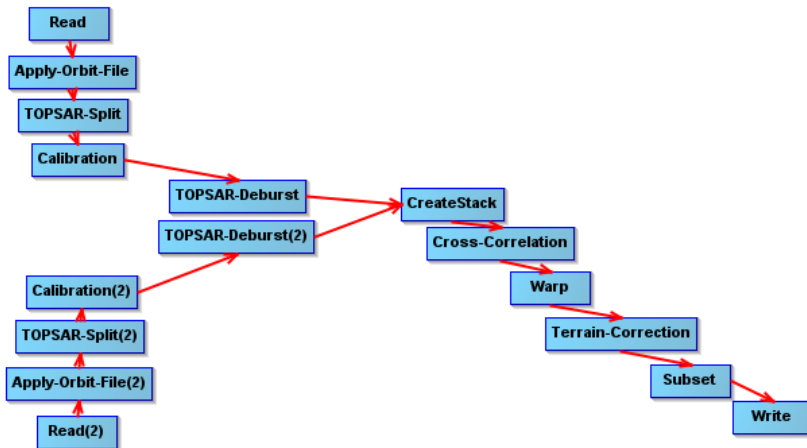
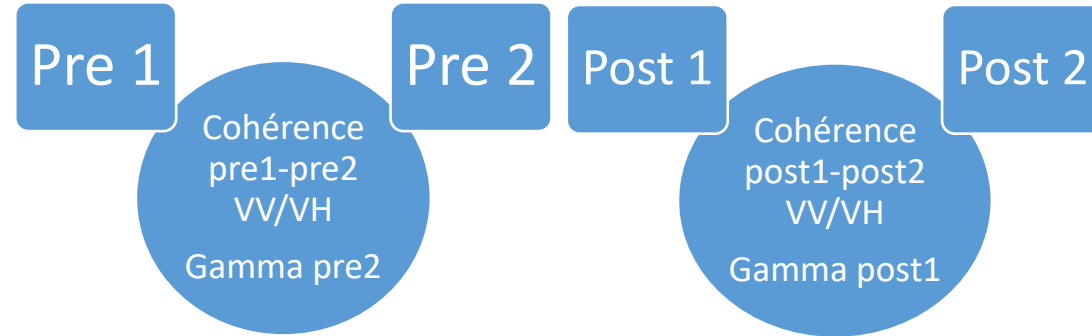
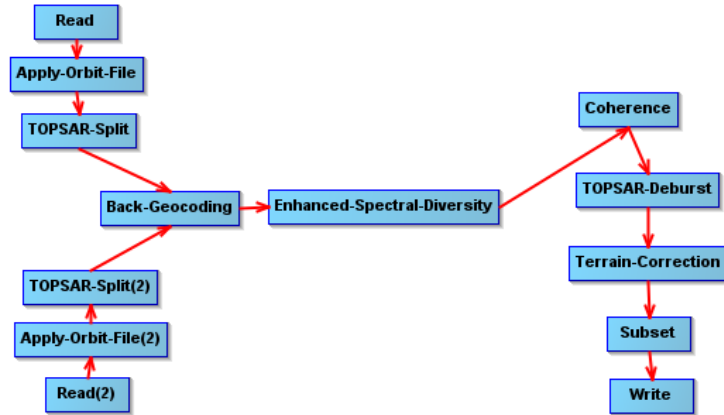
Prétraitements

Objectif double :

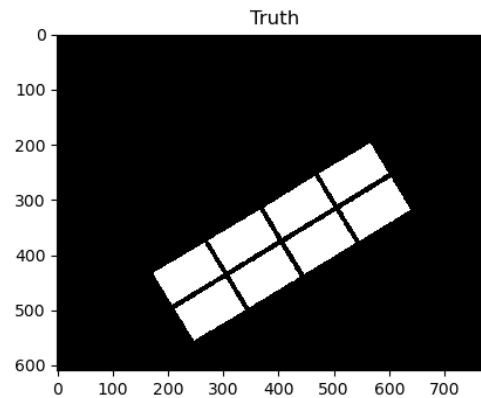
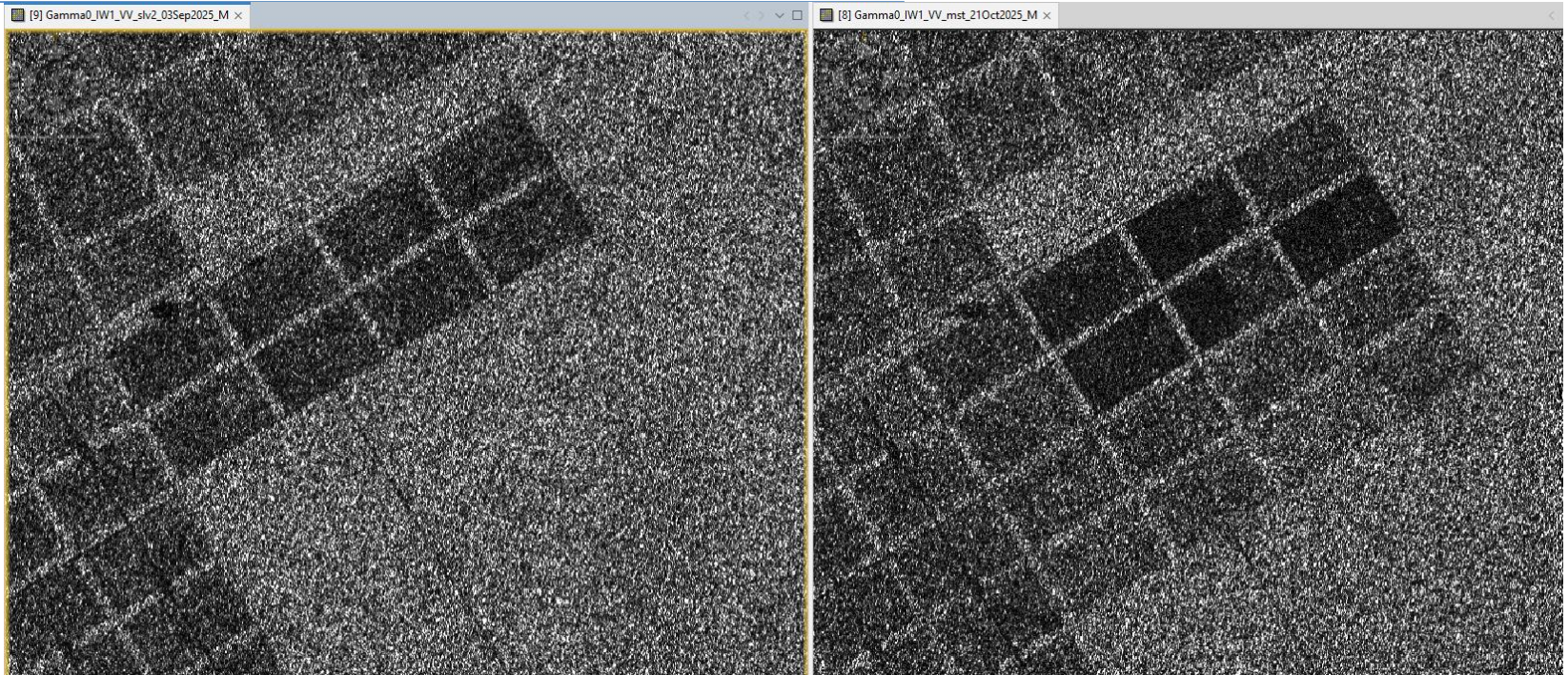
- ❖ Mettre en forme les images: orthorectification, coregistration et éventuellement premières opérations susceptibles de faire ressortir les différences
- ❖ Passer de 4 images SAR à deux produits pre/post événement



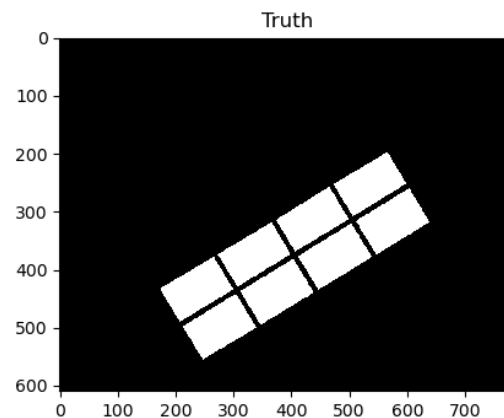
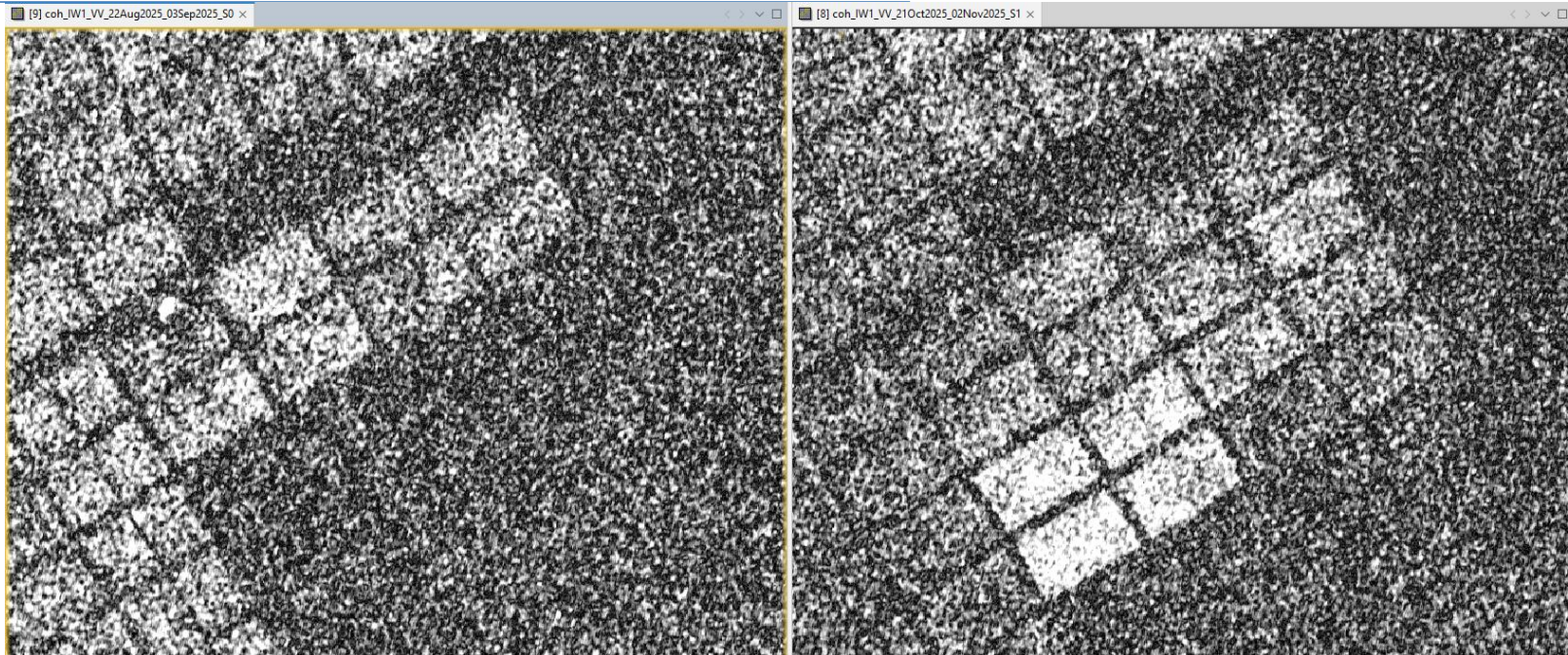
Prétraitements



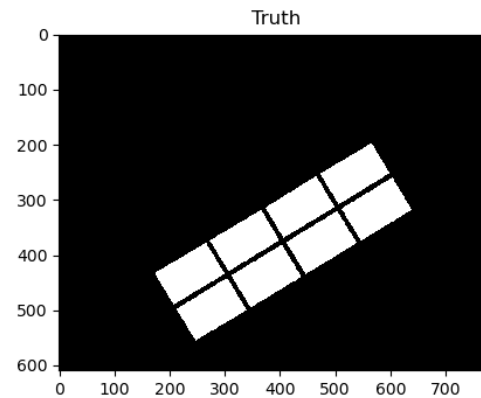
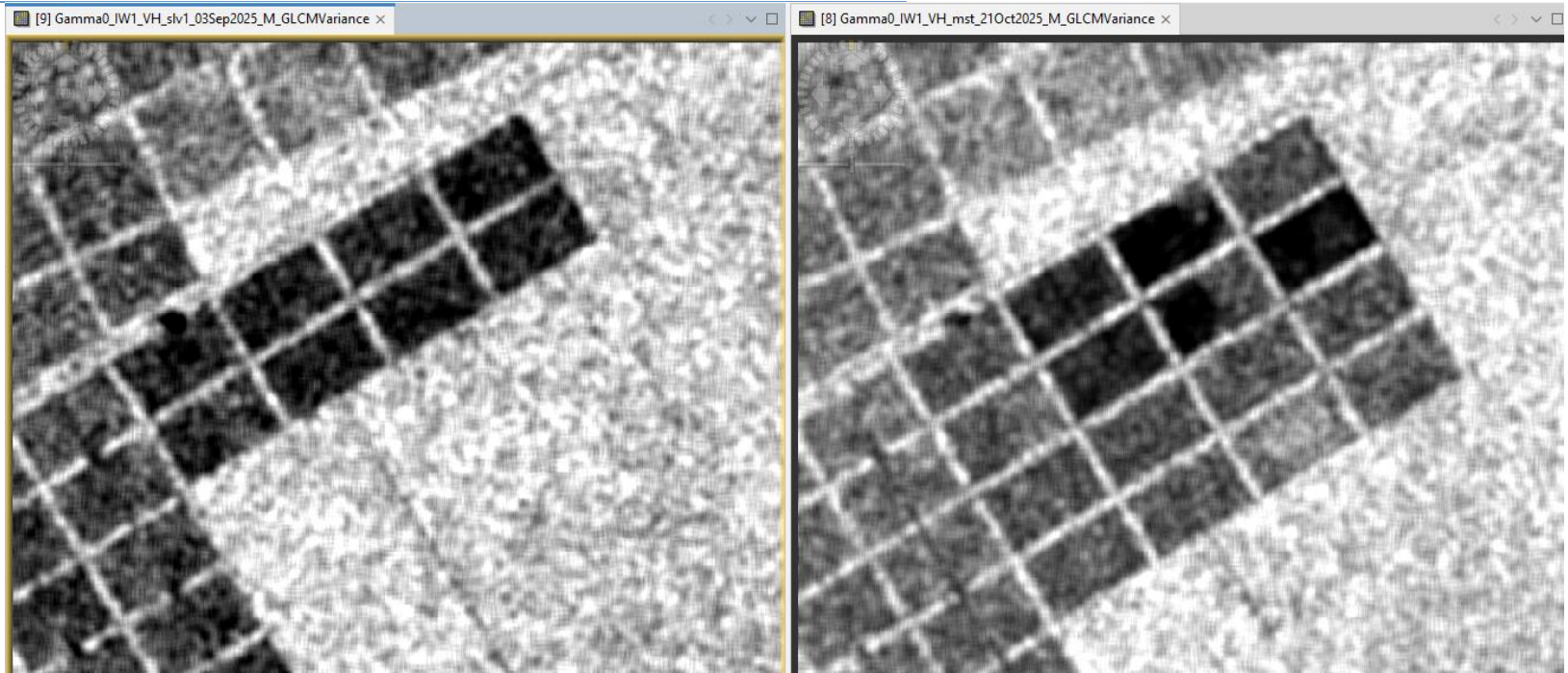
Prétraitements



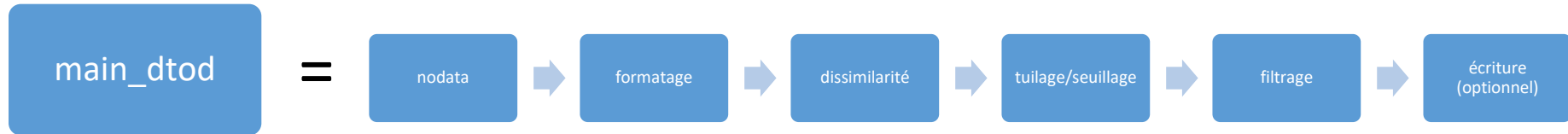
Prétraitements



Prétraitements



Brique main_dtod



```
def main_dtod_0(path_img1: str, path_img2: str, n: int, k: float = 1.0, closing: bool = False, p: int = 30, d: float = 0.5, a: int = 4000, out_path: str | None = None) -> tuple[dict, np.ndarray]:
```

Inputs :

- ❖ chemins vers le couple d'images pre – post (GeoTiff multibandes)
- ❖ `n` : nombre de sous-tuile
- ❖ `k` : coefficient correctif du seuillage
- ❖ `closing`, `p`, `d`, `a` : paramètres du filtre
- ❖ chemin facultatif de l'image binaire de changement (GeoTiff monobande) pour écriture

Outputs :

- ❖ profile de l'image binaire de changement
- ❖ image binaire de changement

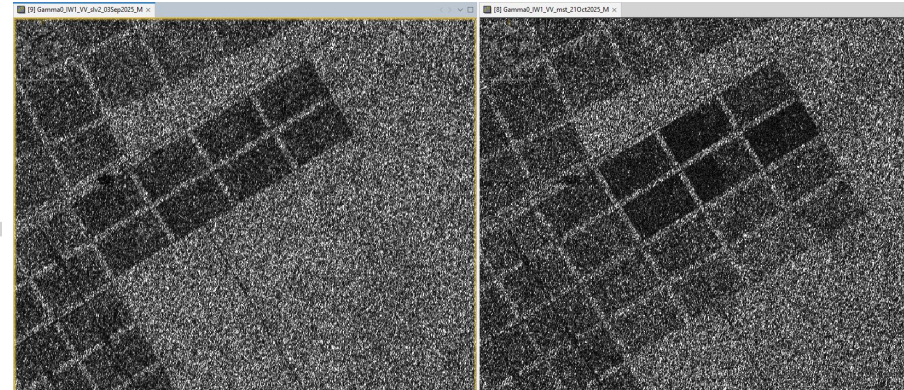
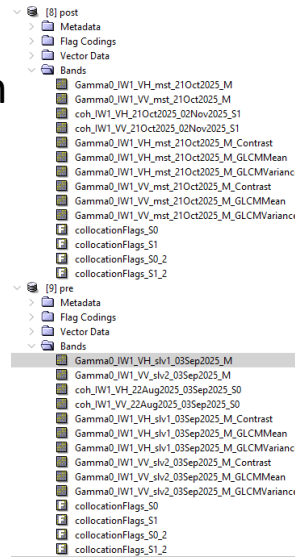


Important de maîtriser le géoréférencement à chaque étape, et de mettre à jour le profil correspondant au masque de sortie pour écrire un raster qui correspond à une réalité géographique

Brique main_dtod

nodata

- ❖ Première étape: récupération des bandes avec rasterio: GeoTiff



- ❖ Gestion des Nodata: différents formats possibles. Se ramener à une convention unique et faire attention à ne pas les propager des les opérations qui suivent

```
Image 1 : (1, 528, 643) bands = 1  
Image 2 : (1, 529, 643) bands = 1  
Number of NaN : 0
```



```
Image 1 : (1, 528, 643) bands = 1  
Image 2 : (1, 529, 643) bands = 1  
Number of NaN : 0
```



nombre de Nan dépend du prétraitement

Brique main_dtod

formatage

- ❖ Exclusion des valeurs extrêmes: on choisit de garder les valeurs entre les percentiles p_low et p_high (par défaut 2 et 98)
- ❖ On normalise indépendamment chaque bande entre 0 et 1 afin que la comparaison de grandeurs physique différentes ait un sens
- ❖ On calcul pour chaque pixel la distance euclidienne entre les vecteurs pre et post composées de leur valeur sur chaque bande. En plus, en fonction de la nature de la bande d'entrée, on essaye d'anticiper l'évolution (on force gammaVH1yx-gammaVH2yx>=0 par ex)

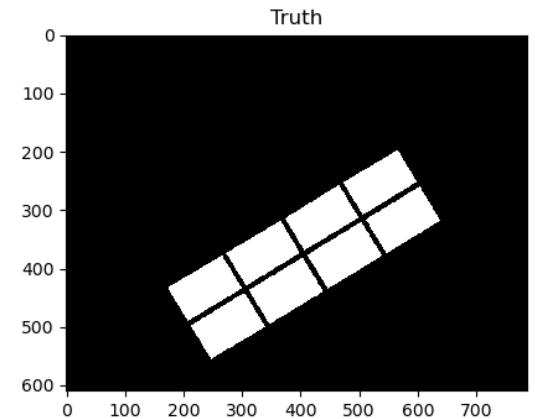
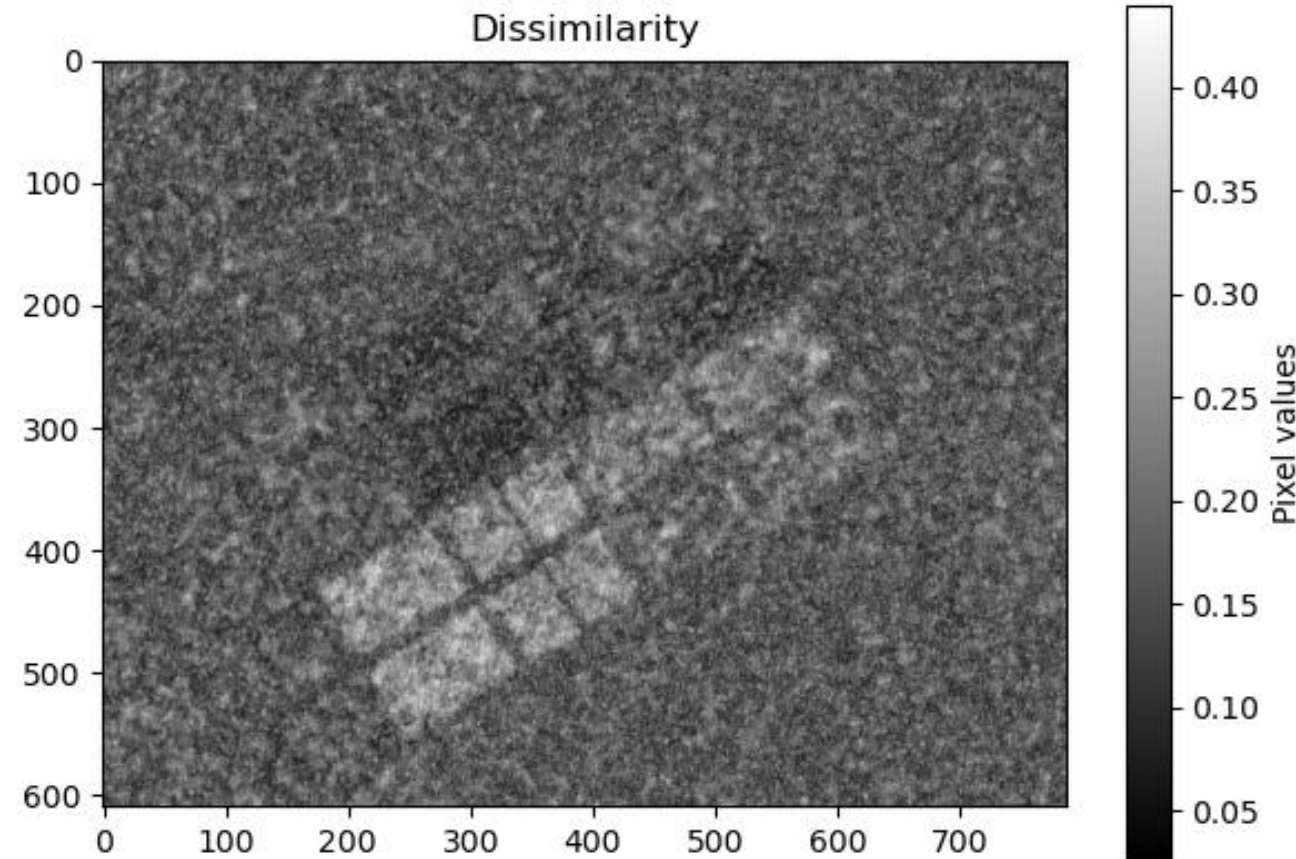
```
53 val_min = np.nanmin(img1)
54 val_max = np.nanmax(img1)
55 print(f"Value min : {val_min}")
56 print(f"Value max : {val_max}")
57
58
[34] ✓ 0.0s
... Value min : 0.0
    Value max : 1.0
```

dissimilarité

$$d(y, x) = \sqrt{\sum_{b=1}^N (img1_{b,y,x} - img2_{b,y,x})^2}$$

Brique main_dtod

dissimilarité



Brique main_dtod

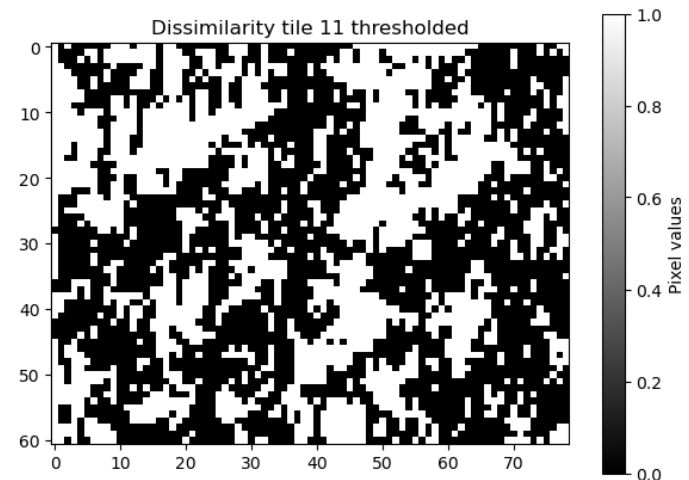
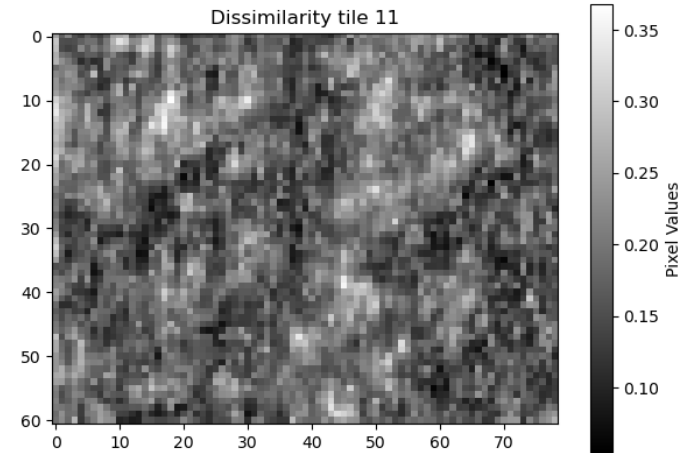
tuilage /
seuillage

❖ Objectif : séparer les pixels qui correspondent à une zone qui a subi un changement / n'a pas subi de changement

-> on ne peut pas avoir un seuil pour l'image entière. Si les zones sont trop hétérogènes par nature on a besoin de seuils différents localement :

- on découpe notre image en amont en sous-tuiles, en « paddant » avec des lignes et colonnes de NaN pour combler les trous
- on seuille par méthode OTSU, ajout d'un coefficient correctif k pouvoir contrôler « manuellement » le nombre de positifs

```
Thresholds : [[0.06412691 0.10207628 0.08401305 0.09814252 0.09737706 0.08209924  
0.09437369 0.10169315 0.07779166 0.11458929]  
[0.0653579 0.08247314 0.08224203 0.09204045 0.06483263 0.08422439  
0.08504701 0.10678199 0.08114009 0.09647547]  
[0.08612427 0.09893315 0.07792124 0.08693326 0.08326148 0.07967756  
0.08399985 0.08095539 0.1032595 0.09894343]  
[0.08322962 0.10838827 0.07803479 0.07829902 0.12359916 0.11827681  
0.07291878 0.07863462 0.07681584 0.09282722]  
[0.06331589 0.14735523 0.07865648 0.09298631 0.13225505 0.13094887  
0.08032726 0.0703652 0.08089081 0.07752934]  
[0.11900384 0.09217885 0.07736815 0.13068931 0.16182454 0.13713071  
0.11266423 0.07696477 0.09890058 0.08579559]  
[0.10937121 0.10336678 0.08737145 0.10768984 0.17604351 0.18057188  
0.08955101 0.09484469 0.10097279 0.08862255]  
[0.08577093 0.09876937 0.09058105 0.13550901 0.1023773 0.0895745
```



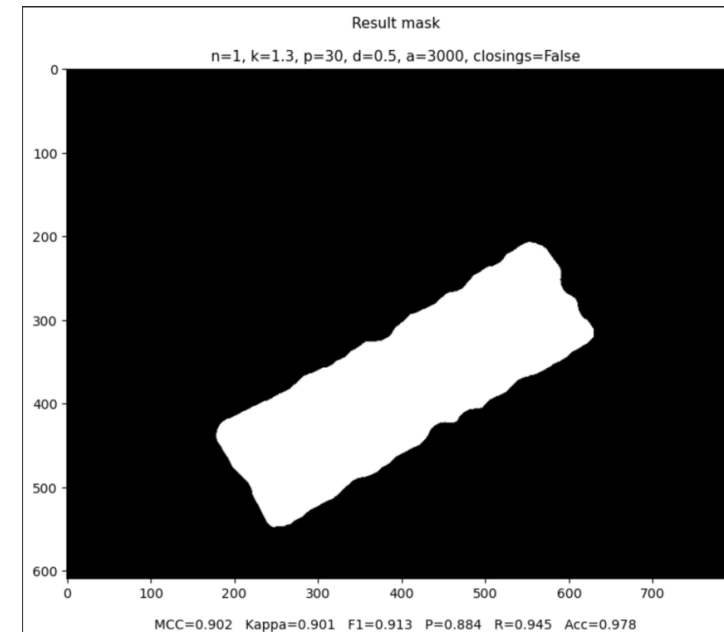
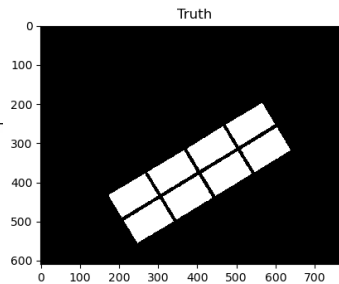
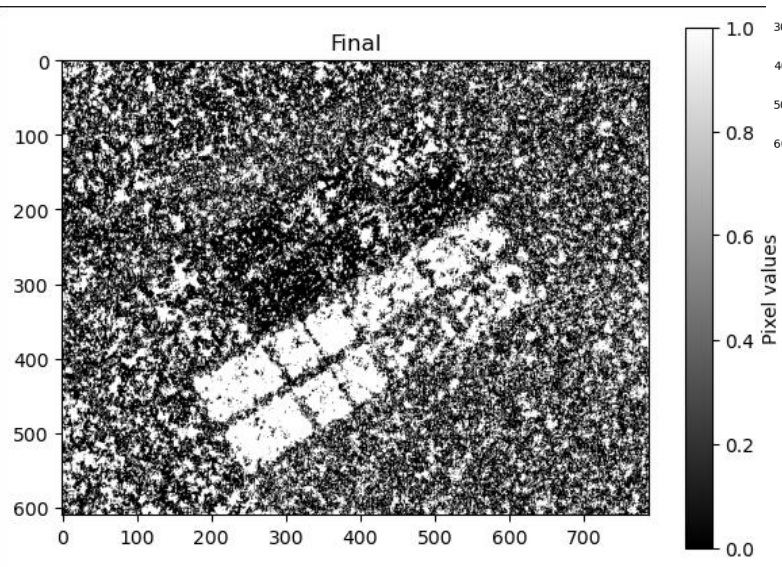
Brique main_dtod

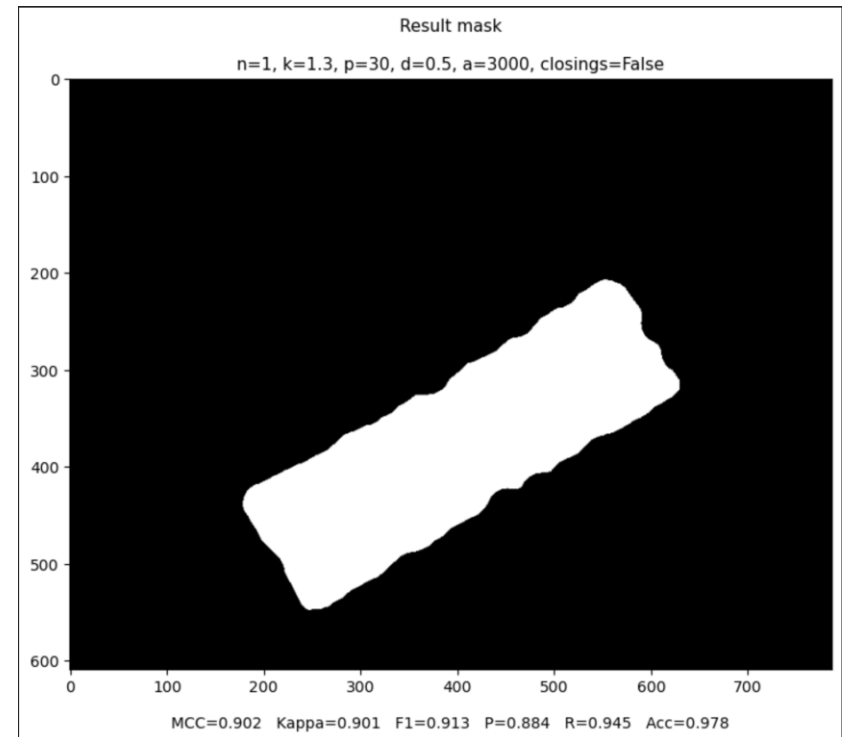
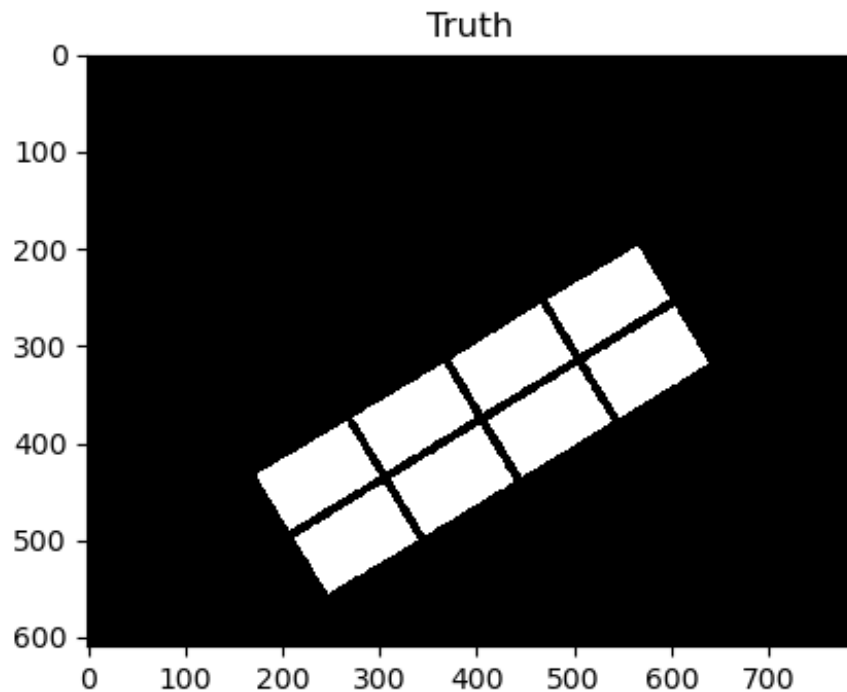
filtrage

- ❖ Image obtenue après concaténation des sous-tuiles présente souvent un effet sel et poivre avec des zones plus denses là se sont produits les changements



on applique un filtre de densité, puis on ne garde que les zones connectées d'une surface minimale, et éventuellement un filtre closing en fonction de ce qu'on veut



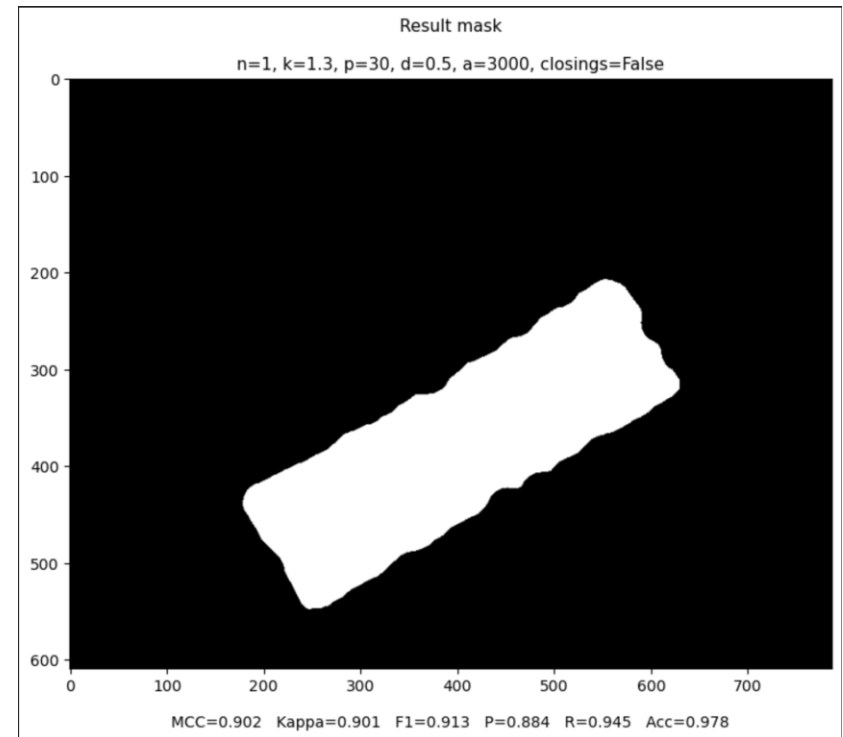


Brique main_dtod

écriture
(optionnel)

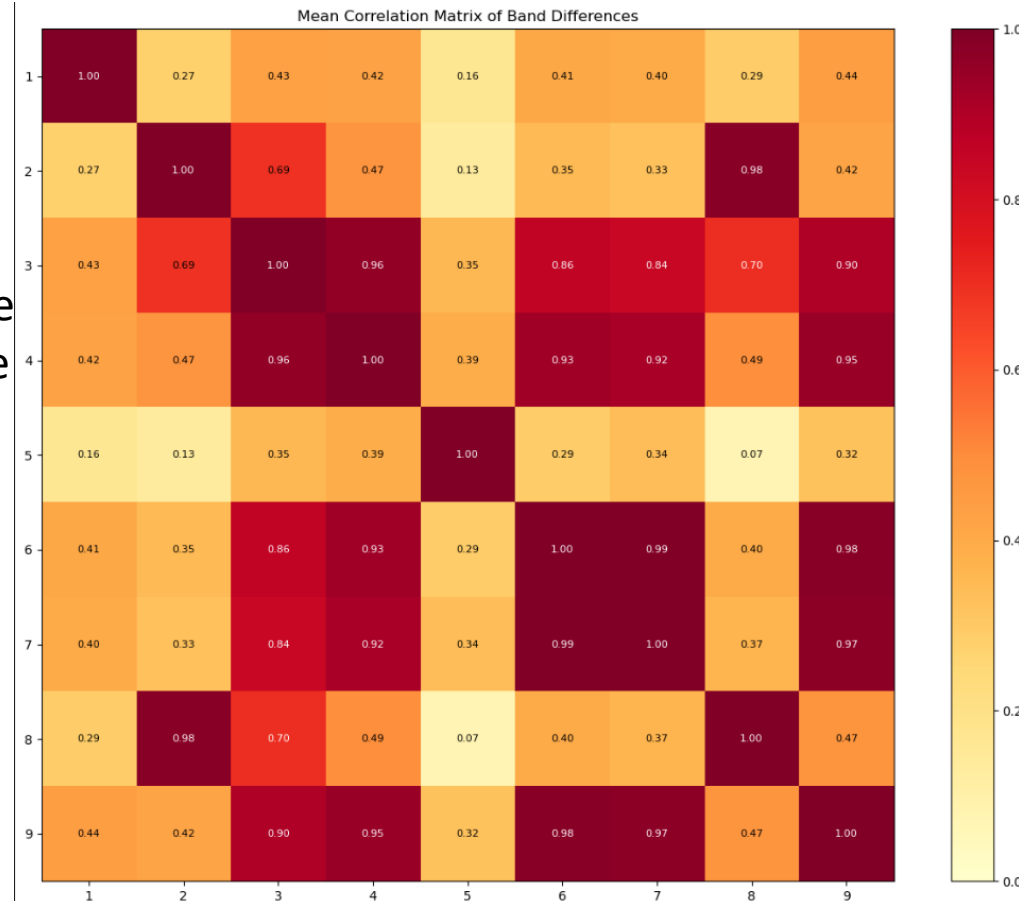
- ❖ Le `main_dtod` renvoie le profile, l'image et peut écrire le GeoTiff à la demande de l'utilisateur. Le profile de sortie est alors le même que celui de l'image 1 donnée au `main_dtod`, à l'exception des nodata qui valent 0 et du type de donnée qui est `uint8`.

```
Height: 528
{'blockysize': 528,
 'count': 1,
 'crs': CRS.from_epsg(32721),
 'driver': 'GTiff',
 'dtype': 'uint8',
 'height': 528,
 'interleave': 'band',
 'nodata': 0,
 'tiled': False,
 'transform': Affine(10.0, 0.0, 441765.63746730745,
                    0.0, -10.0, 8982078.826802952),
 'width': 643}
```

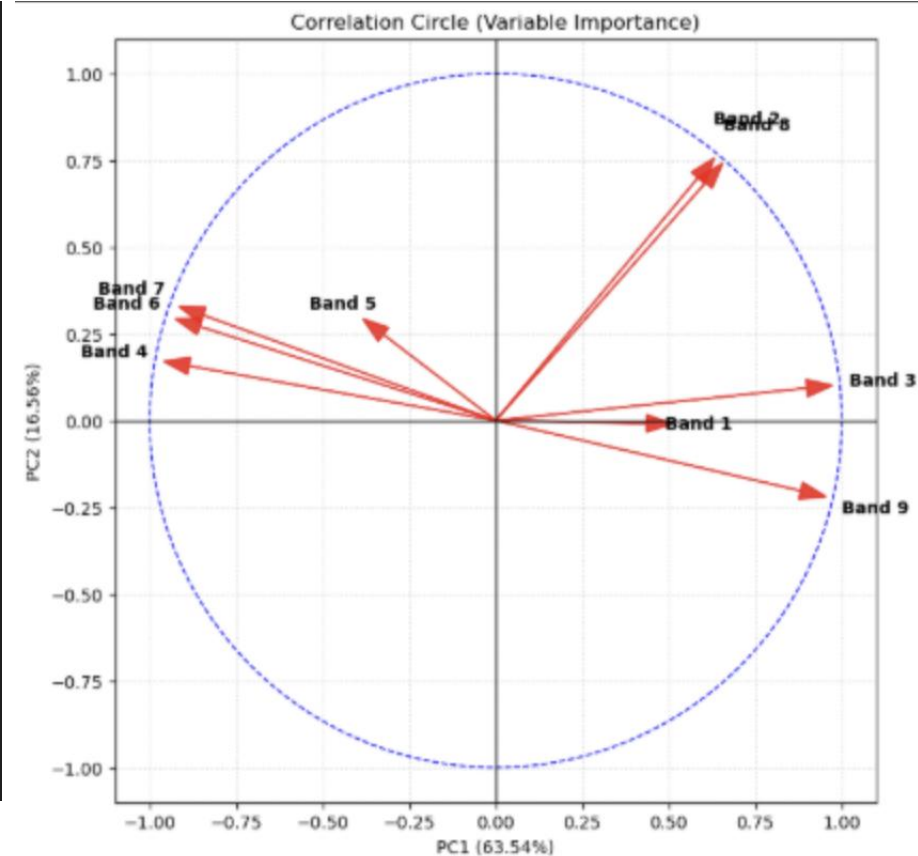
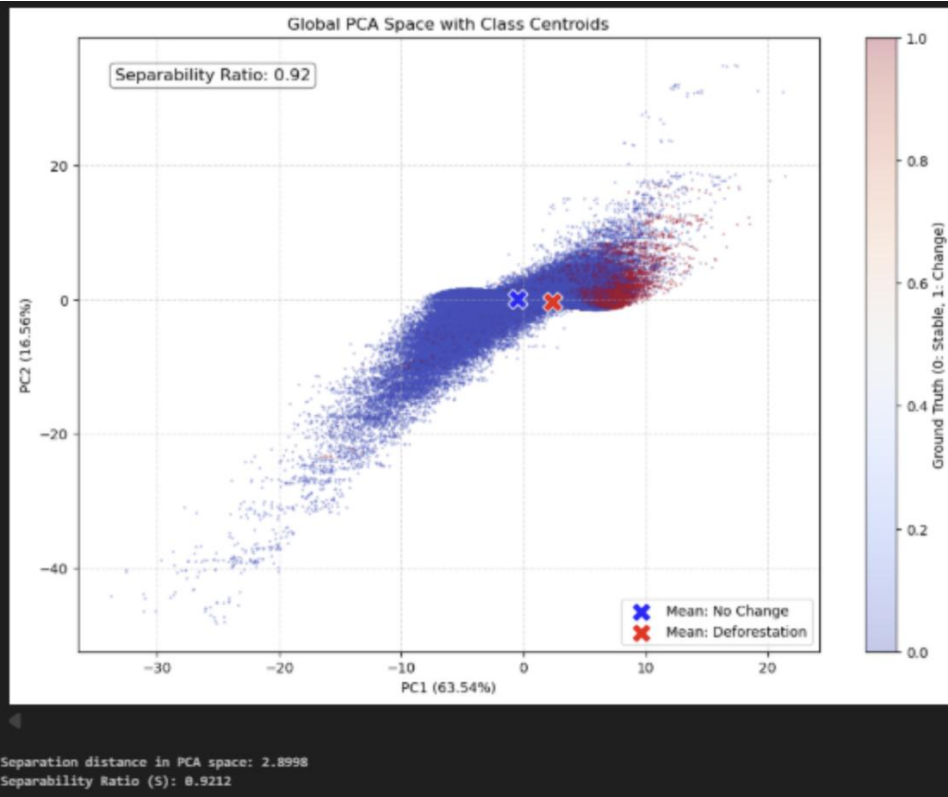


Choix des features

- ❖ Utilisation des textures de Haralick : quantification de la **corrélation spatiale** entre les niveaux de gris des pixels voisins à l'aide d'une **matrice de cooccurrence (GLCM)**
- ❖ Utilisation d'indices dérivés, ex : Radar Vegetation Index **$RVI = 4 * VH / (VV + VH)$**
- ❖ Au total, 90 features différentes en entrée impossible de choisir à la main. On réalise une étude statistique à partir de 2 outils : un tableau des corrélations, et une PCA (Principal Component Analysis)



Choix des features



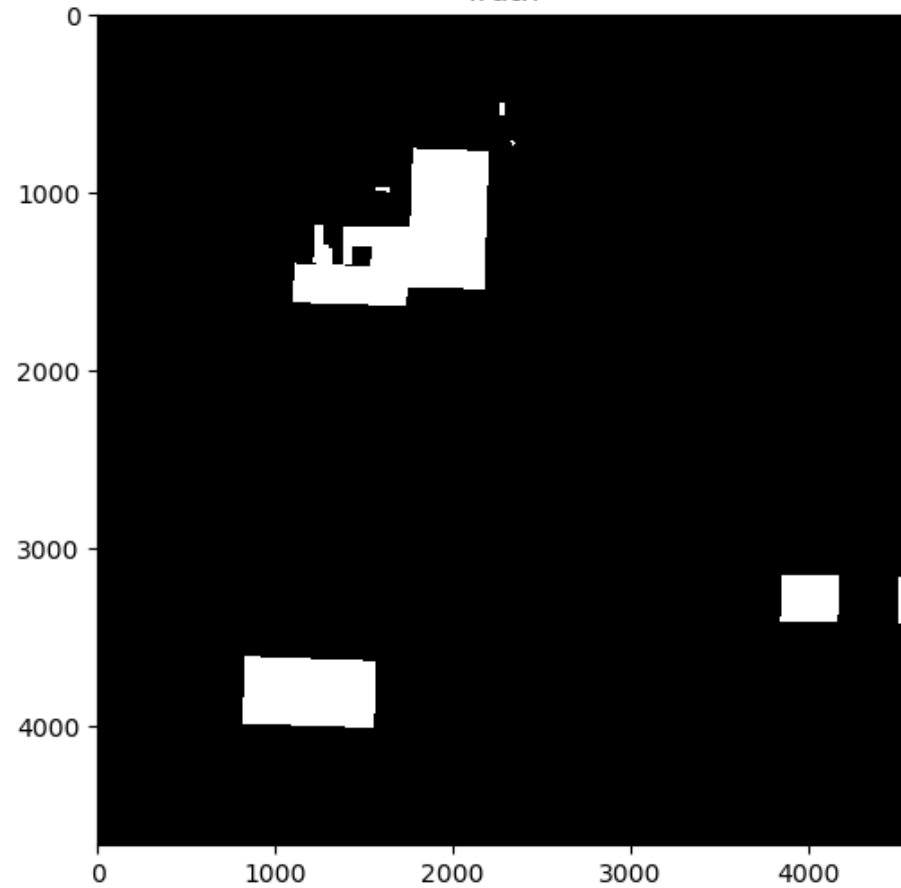
- ❖ Deux informations importantes : les classes sont-elles séparables (deux critères) , et si oui comment

Résultats



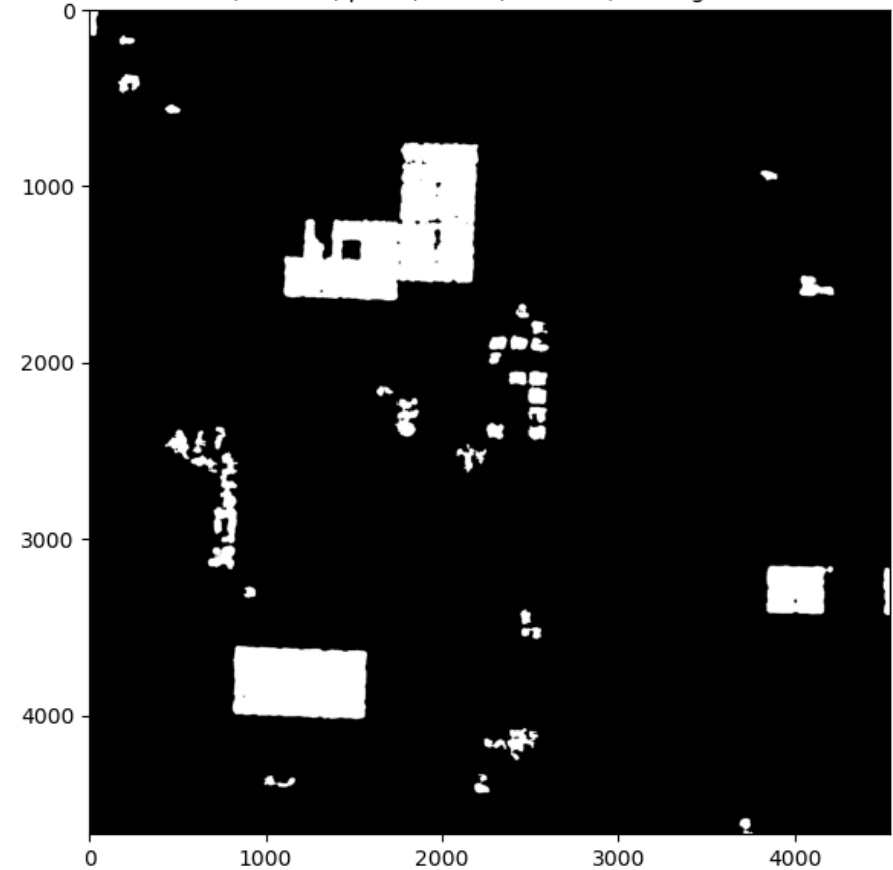
Résultats

Truth



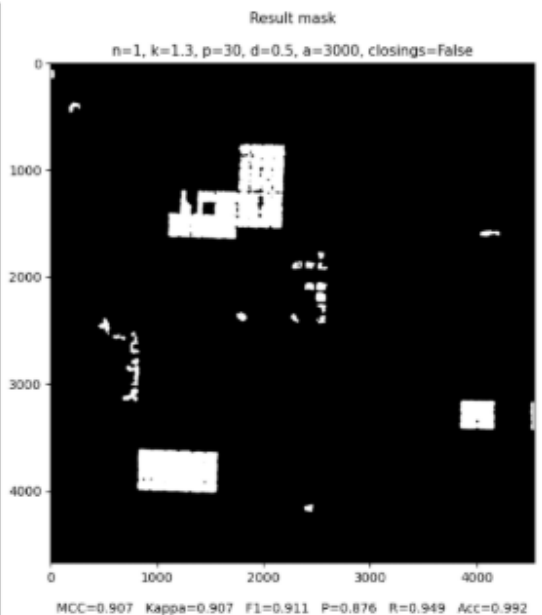
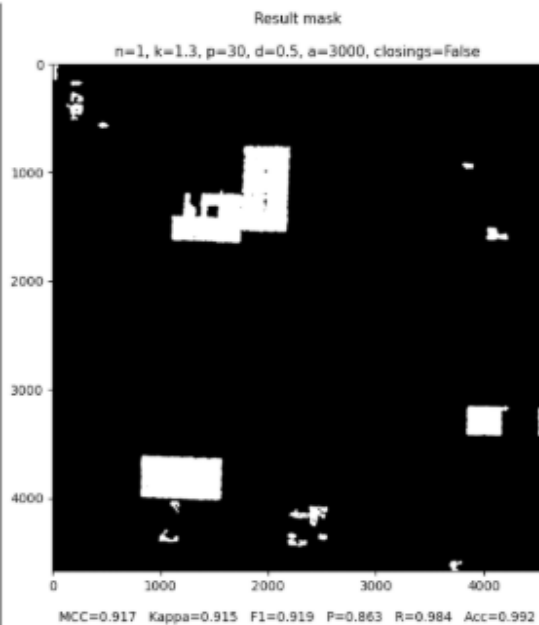
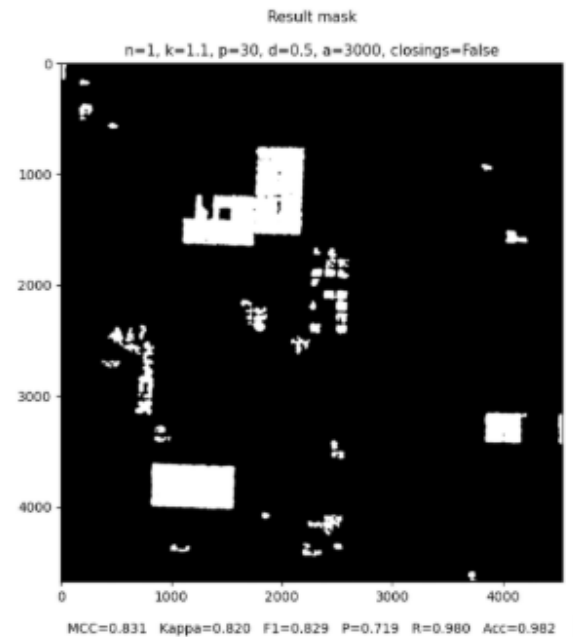
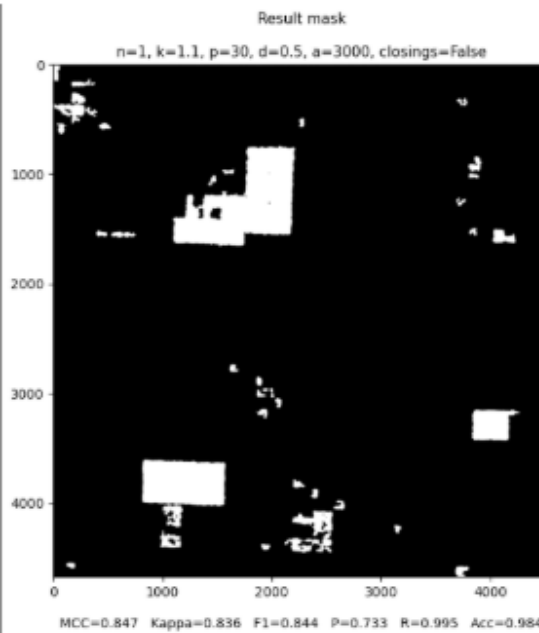
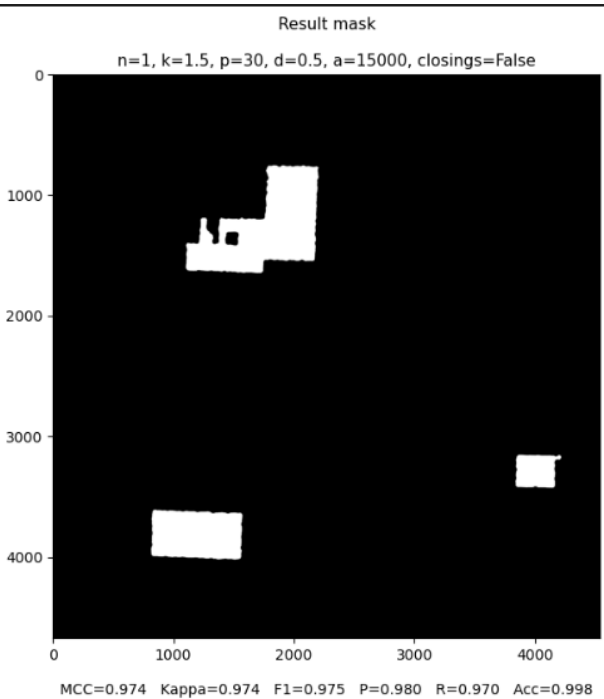
Result mask

$n=1$, $k=1.15$, $p=30$, $d=0.5$, $a=3000$, closings=False

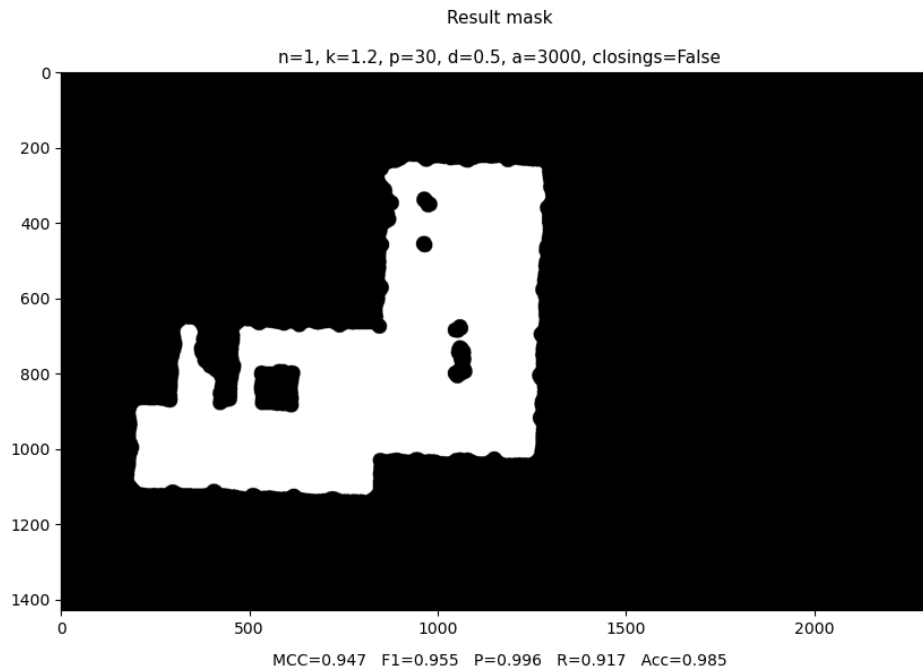
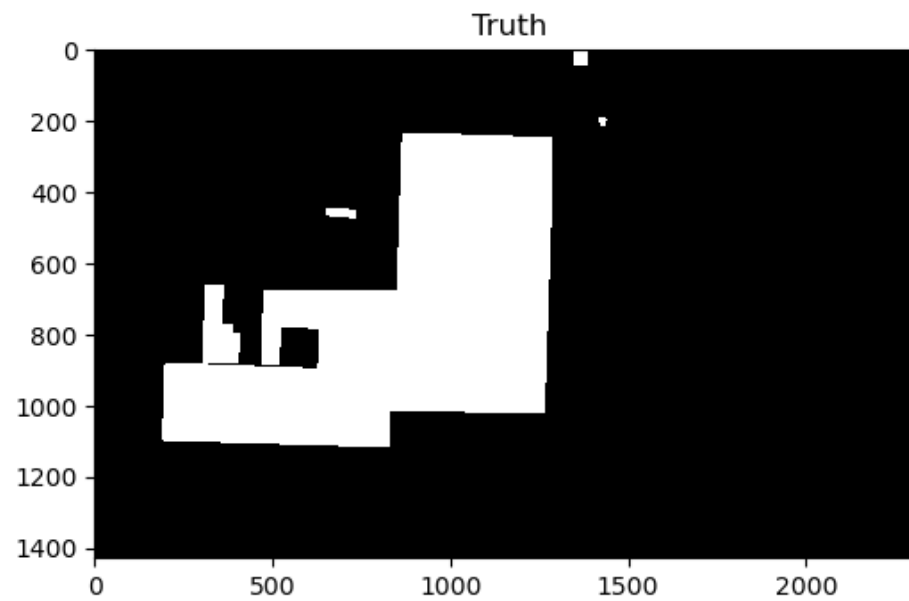


MCC=0.857 F1=0.857 P=0.765 R=0.975 Acc=0.986

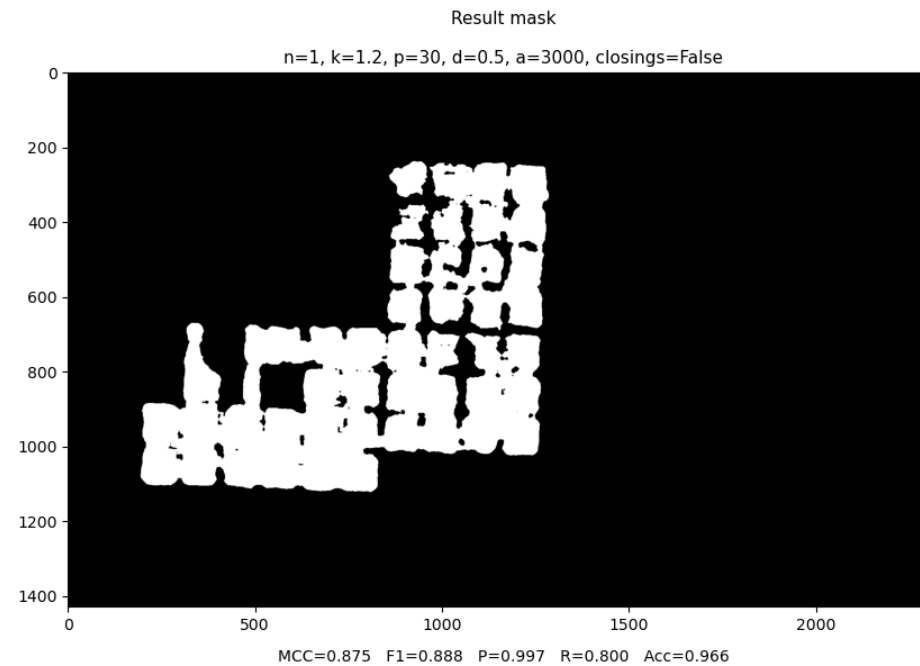
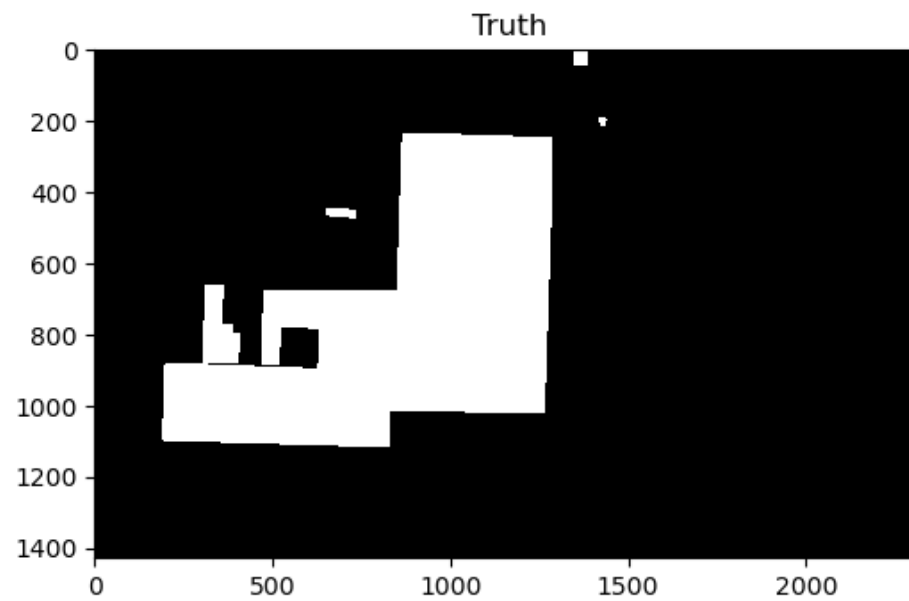
Résultats



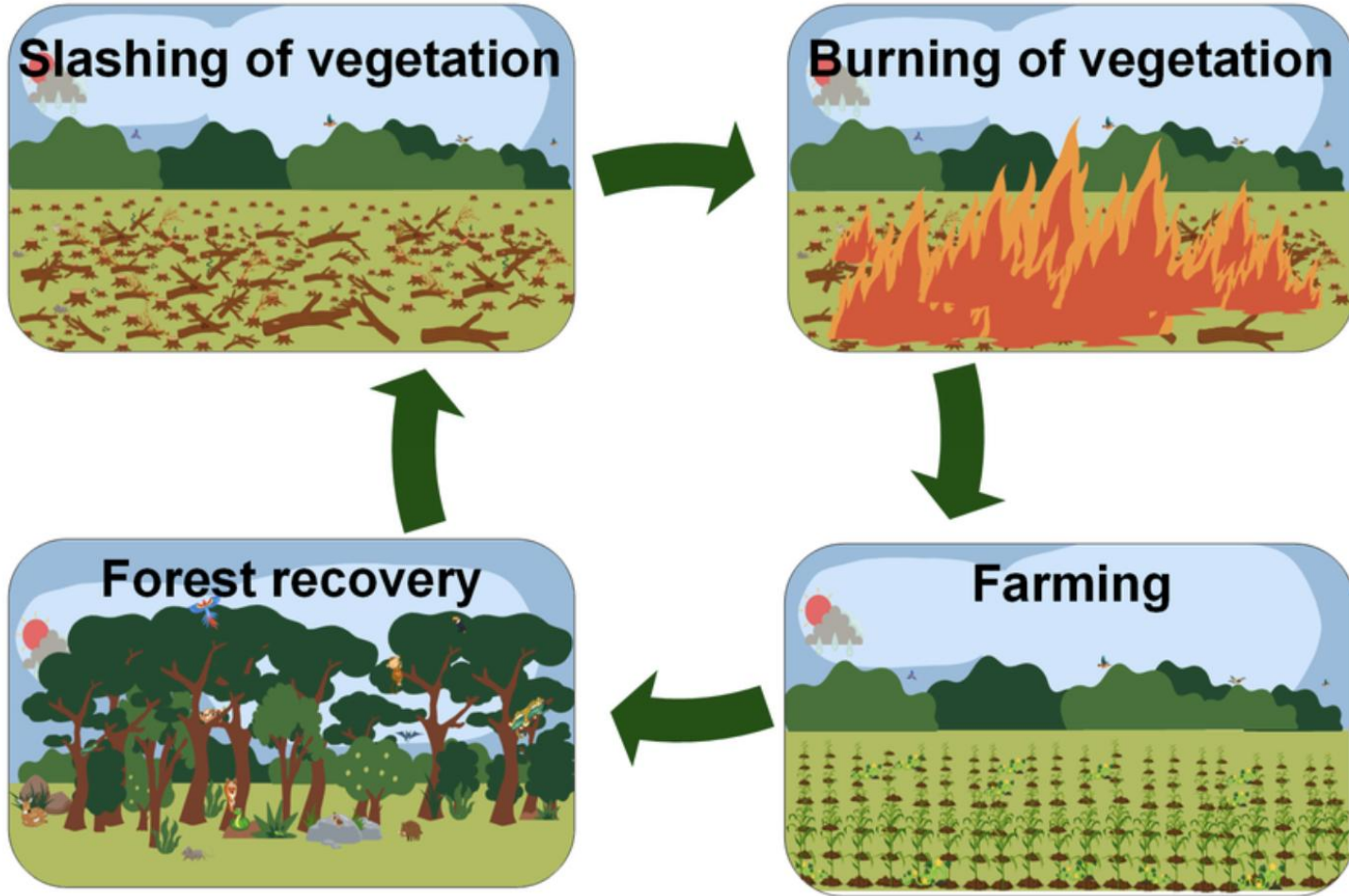
Résultats



Résultats

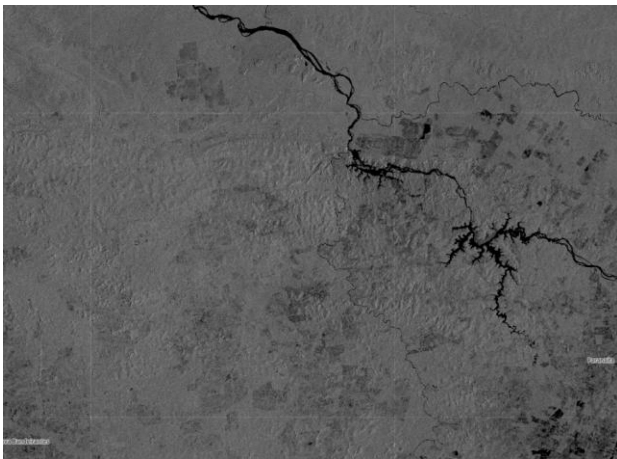


Cycles

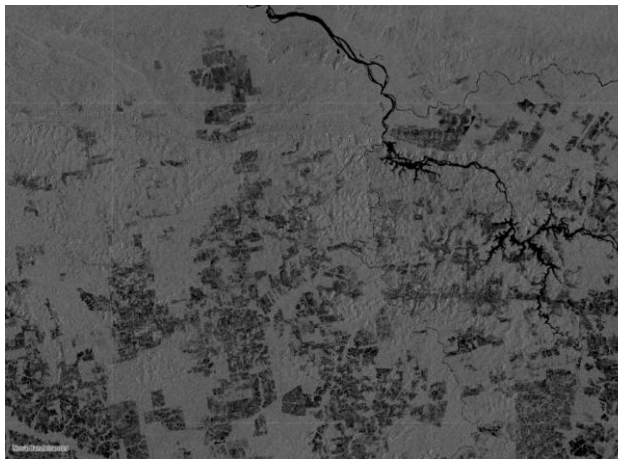


Saisonnalité

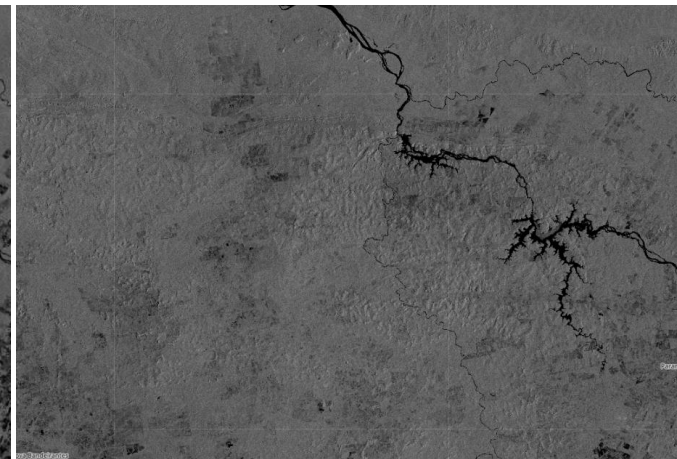
- ❖ Problème : au fil des saisons le sol est plus ou moins humide, et la rétrodiffusion change. Il peut arriver que la rétrodiffusion du sol nu soit alors similaire à celle de la canopée : on détecte alors le changement de surface non pas au moment où il se déroule mais au moment où le sol sèche



Novembre 2018



Juillet 2019



Novembre 2019

Conclusions

- ❖ Cohérence trop sensible
- ❖ La bande C n'est peut-être pas la meilleure, très sensible à la petite végétation.
- ❖ Application possible : surveillance d'une grande zone, on prend un masque de départ pour occulter les zones agricoles changeantes. Ceci permet de garantir un meilleur seuillage et donc une meilleure détection. On actualise ensuite le

Annexes

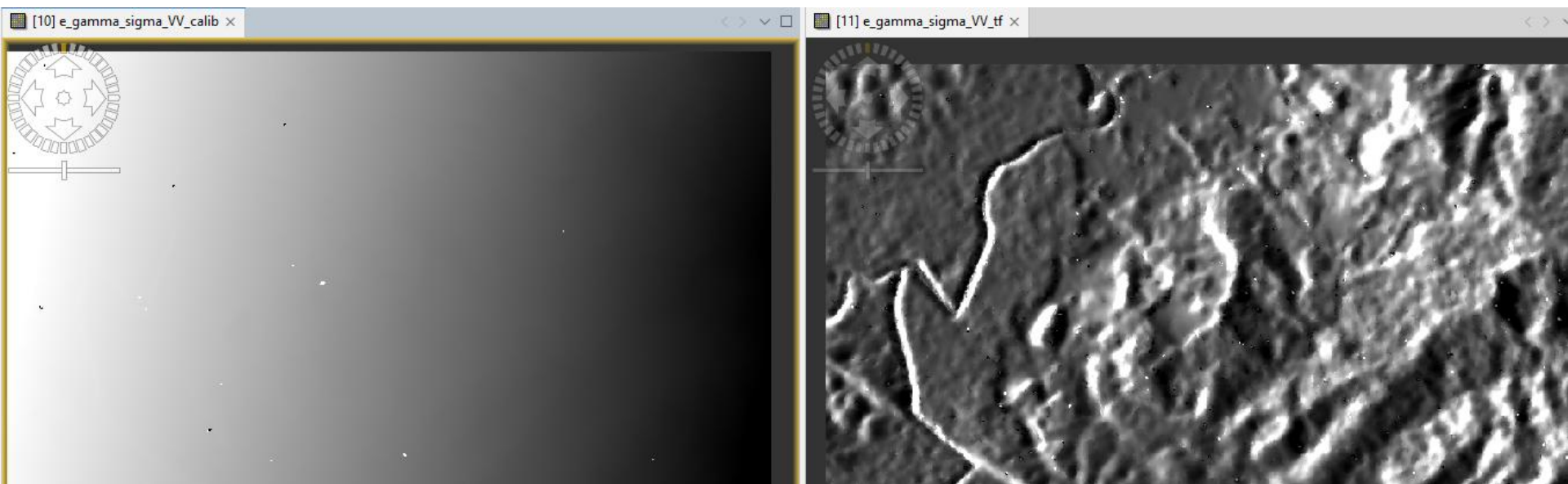
Perspectives d'améliorations

- Régler :
- ❖ #Régler les percentiles que l'on garde
 - ❖ **#Régler le filtre**
 - ❖ Rééchantillonner les S2 pour avoir des shapefile plus précis
 - ❖ Gérer l'imprécision à droite/en bas de l'image due au padding du découpage
 - ❖ #Prendre en compte un **modèle numérique de terrain**
- Tester :
- ❖ Autres pipelines de prétraitement : #meilleur orthorectification, choix de bandes
 - ❖ #Introduction de la **cohérence**
 - ❖ Mieux exploiter le fait **d'étudier plusieurs bandes**
 - ❖ Tester d'autres dissimilarités
 - ❖ **Découper automatiquement** sur des critères statistiques
 - ❖ #Seuiller avec deux gaussiennes / multi OTSU
 - ❖ **#Essayer d'autres filtres**
 - ❖ #Utiliser plusieurs images pre et post même pour la détection date – date
 - ❖ A plus long terme, utilisation de bande L
- Finaliser :
- ❖ #Tester l'algo dans le plus de configurations possibles
 - ❖ **Automatiser** ce qui ne l'est pas
 - ❖ **Monitoring sur le long terme**
 - ❖ Une fois l'algo plus stable, le **décliner en versions** en fonction de l'usage

Perspectives d'améliorations

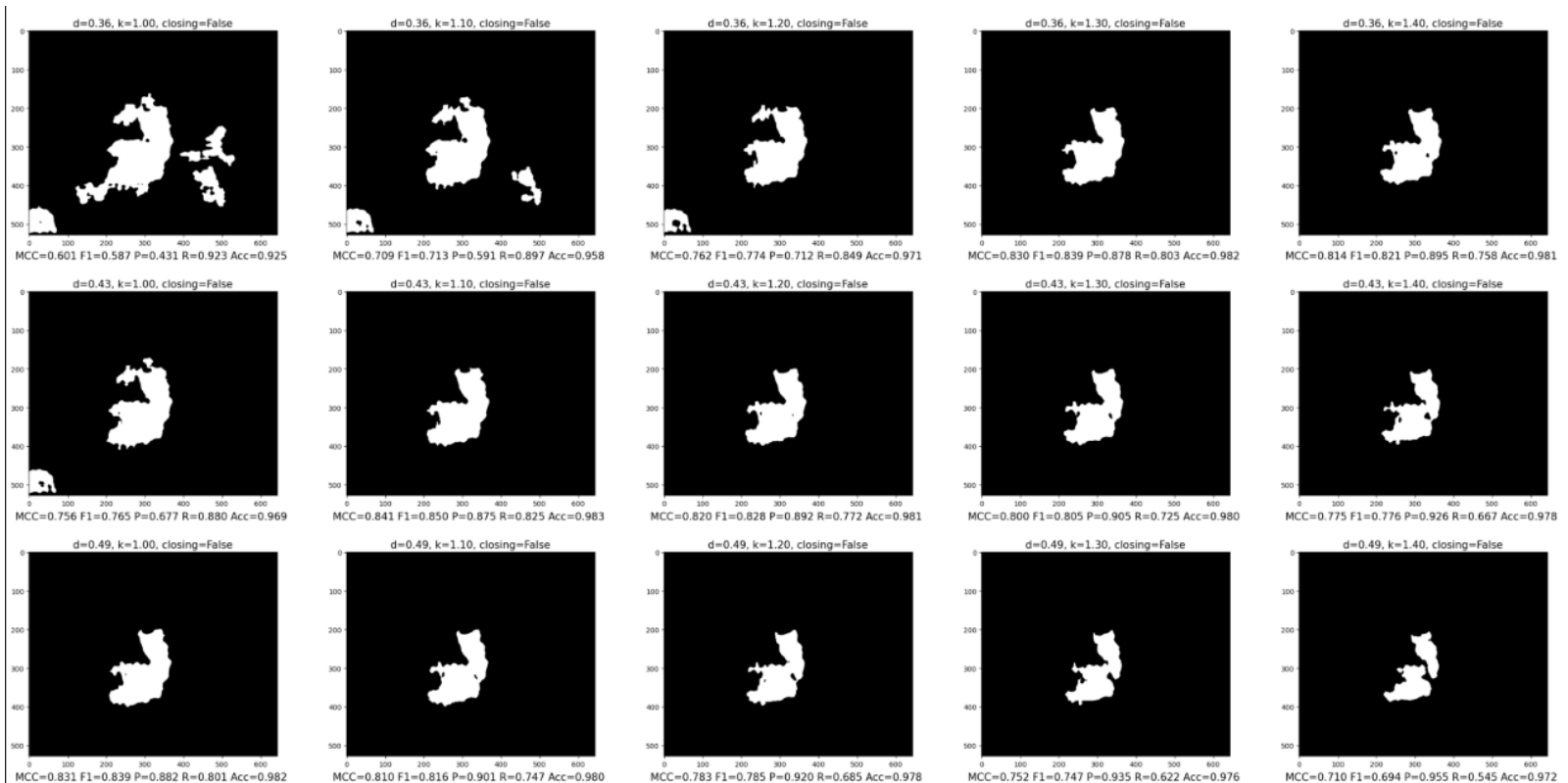
- Régler :
- ❖ #Régler les percentiles que l'on garde
 - ❖ **#Régler le filtre**
 - ❖ Rééchantillonner les S2 pour avoir des shapefile plus précis
 - ❖ Gérer l'imprécision à droite/en bas de l'image due au padding du découpage
 - ❖ #Prendre en compte un **modèle numérique de terrain**
- Tester :
- ❖ Autres pipelines de prétraitement : #meilleur orthorectification, choix de bandes
 - ❖ #Introduction de la **cohérence**
 - ❖ Mieux exploiter le fait **d'étudier plusieurs bandes**
 - ❖ Tester d'autres dissimilarités
 - ❖ **Découper automatiquement** sur des critères statistiques
 - ❖ #Seuiller avec deux gaussiennes / multi OTSU
 - ❖ **#Essayer d'autres filtres**
 - ❖ #Utiliser plusieurs images pre et post même pour la détection date – date
 - ❖ A plus long terme, utilisation de bande L
- Finaliser :
- ❖ #Tester l'algo dans le plus de configurations possibles
 - ❖ **Automatiser** ce qui ne l'est pas
 - ❖ **Monitoring sur le long terme**
 - ❖ Une fois l'algo plus stable, le **décliner en versions** en fonction de l'usage

Gamma ?



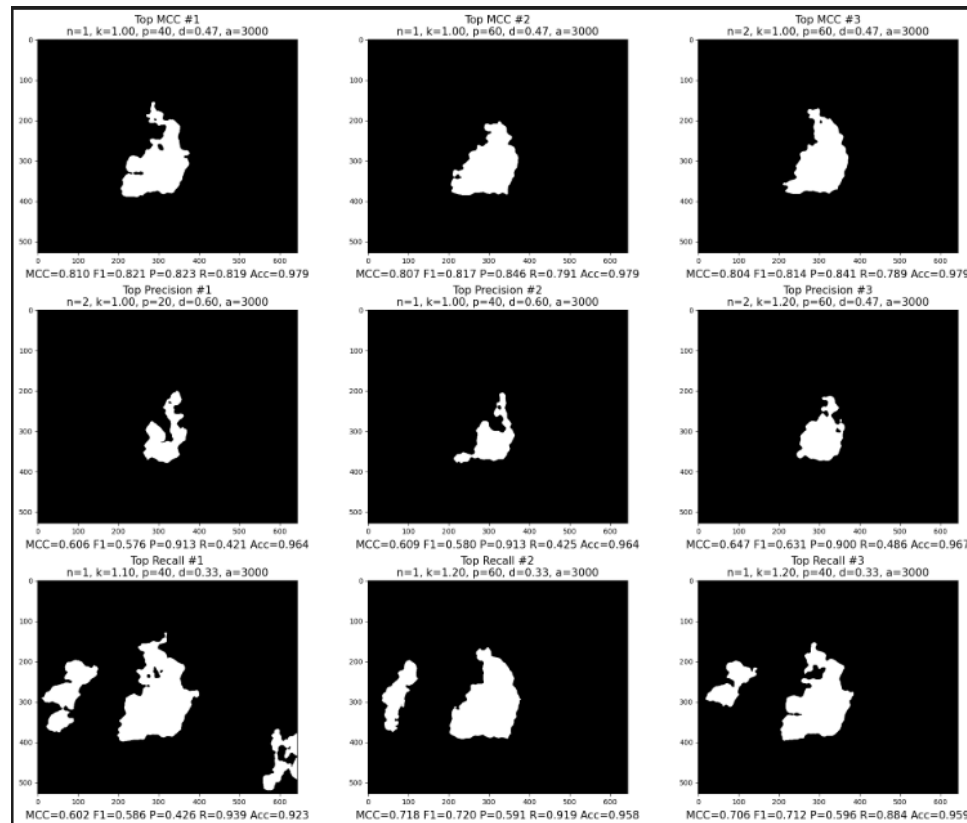
Fonctions test

- ❖ Plusieurs paramètres à tester, pour le filtre, pour le découpage, pour le seuillage :
- 1^{ère} fonction test : à défaut de pouvoir visualiser l'impact de tous ces paramètres simultanément, fonction qui prend en argument deux listes de valeurs pour deux paramètres et affiche un tableau à double entrée pour chaque couple possible



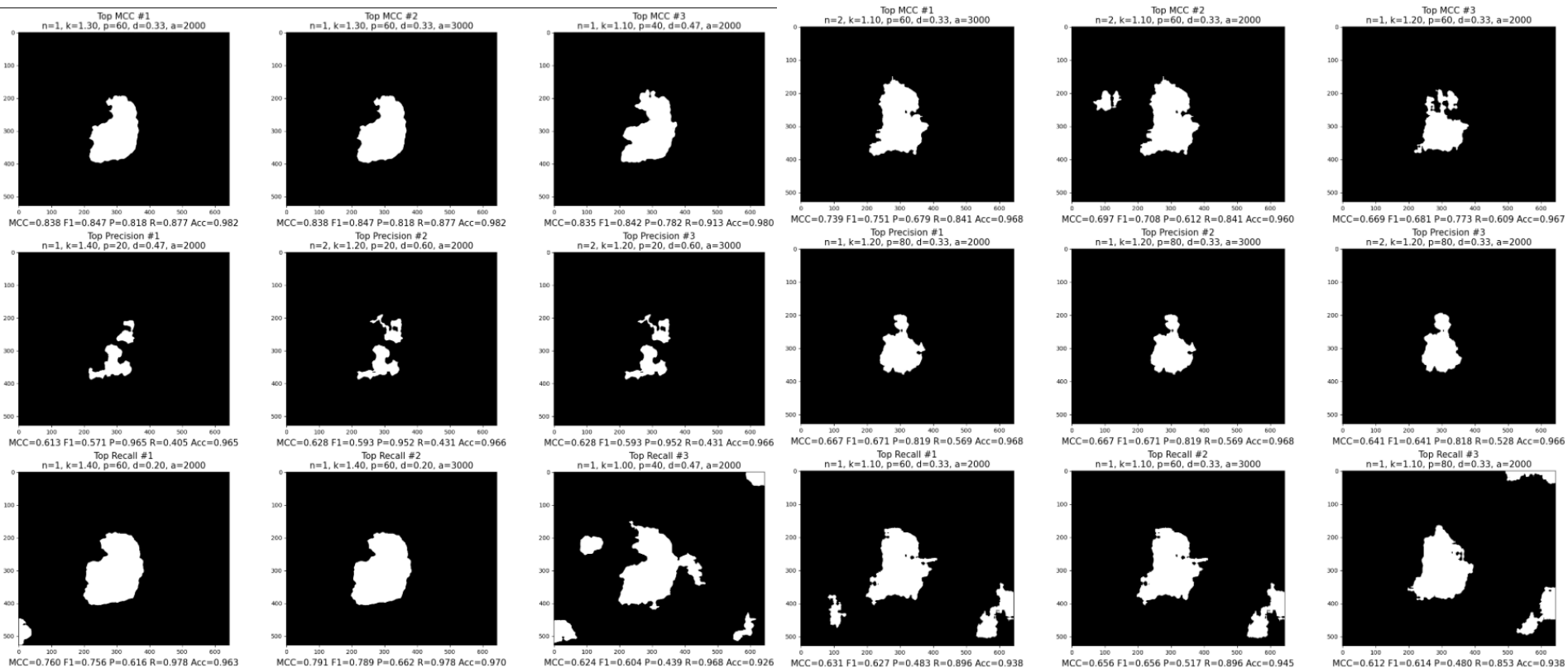
Fonctions test

- 2^{ème} fonction test : fonction qui prend en argument une liste de valeur pour chaque paramètre, calcule les performances pour chaque et retient les trois meilleurs tuples pour le MCC, la précision et le rappel respectivement. Pour ces deux derniers on contrôle que le MCC soit correct



Résultats

❖ Zone t3bis :



8 bandes

Sigma VH